

CS4225/CS5425 Big Data Systems for Data Science

Graphs and PageRank

Ai Xin
School of Computing
National University of Singapore
aixin@comp.nus.edu.sg



School of Computing¹

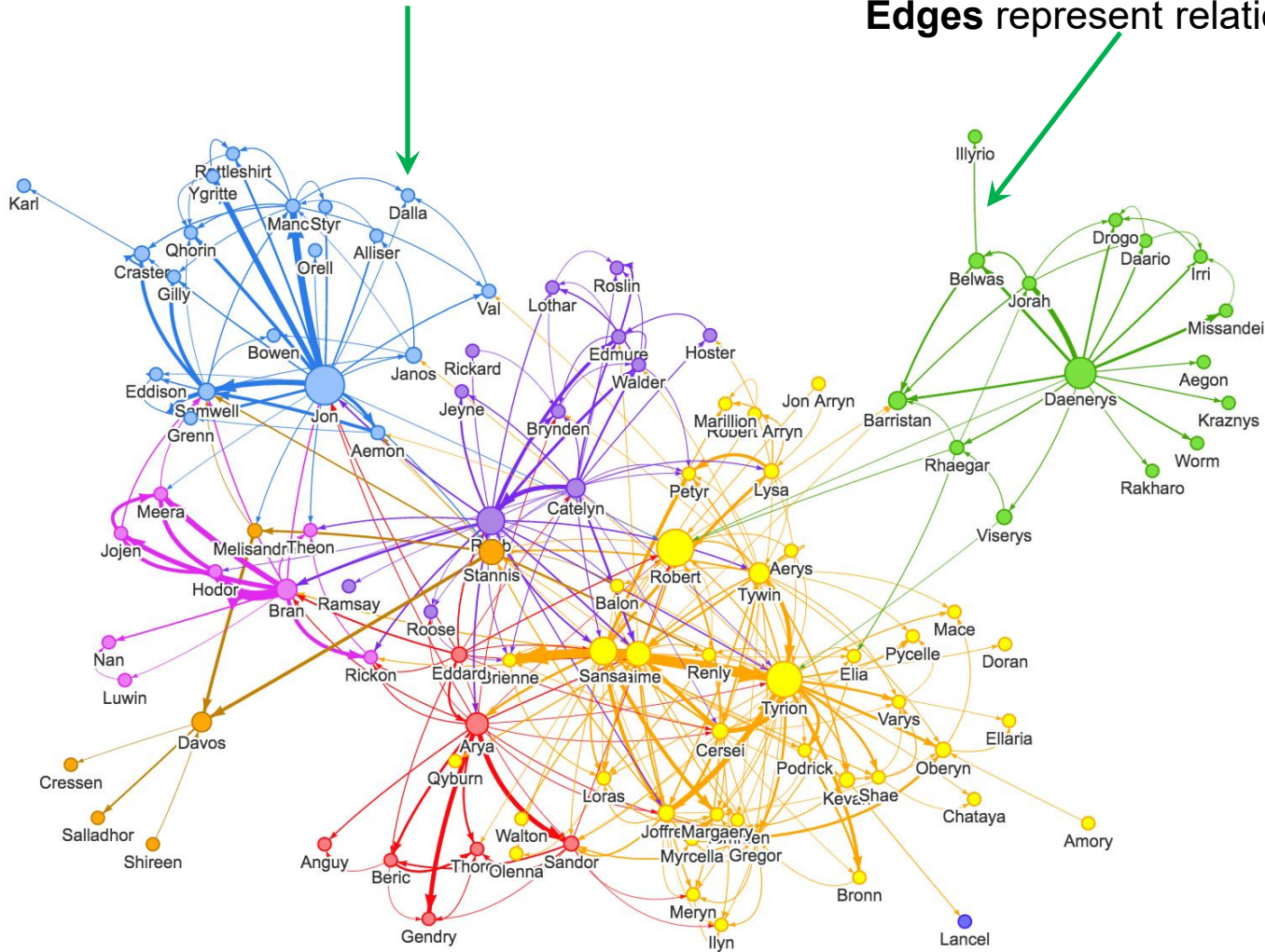
Today's Plan

- Graphs: Introduction
- Simplified PageRank
 - Flow Formulation
 - Random Walk Formulation
- PageRank with Teleports
- Topic Sensitive PageRank
- PageRank Implementation

Graph Data

Nodes (e.g. people)

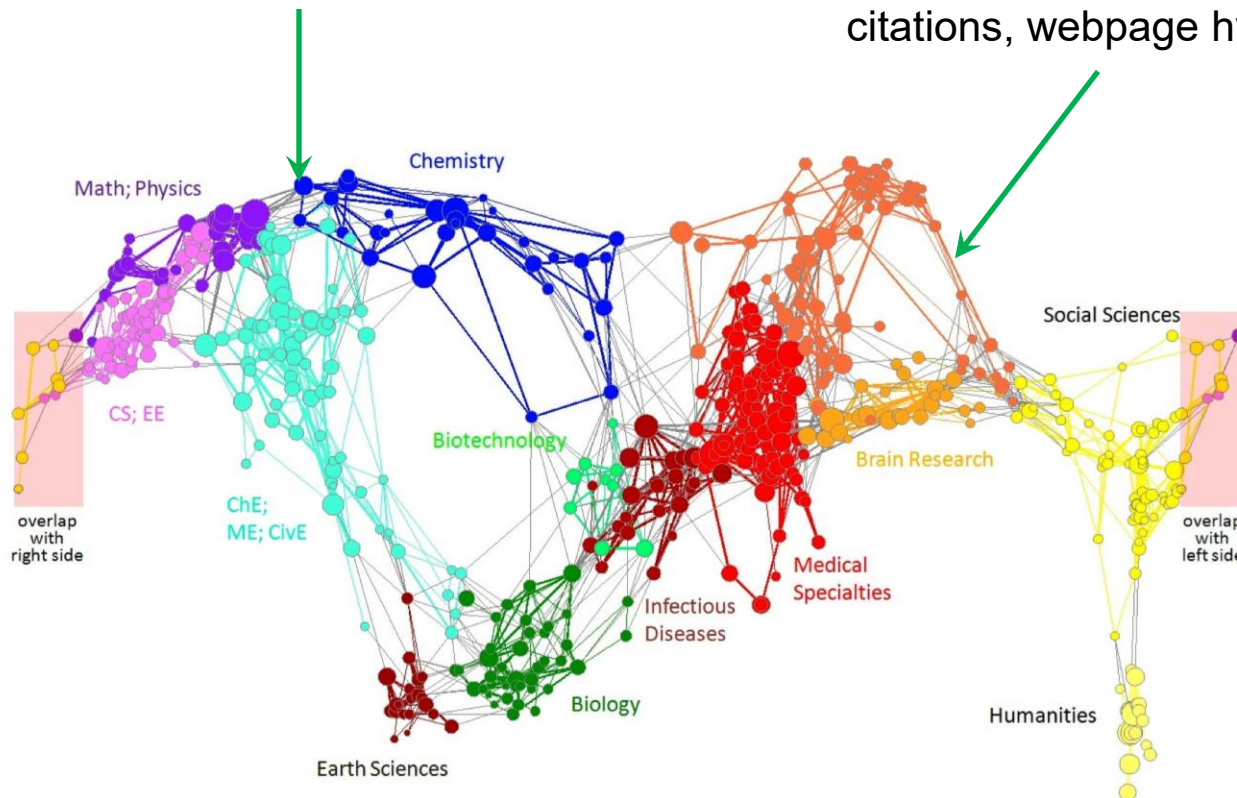
Edges represent relationships



Graph Data: Information Networks

Nodes represent objects
(in this case, journals)

Edges represent relationships
(in this case, citations)
- Can be *undirected* (e.g. friendship) or *directed* (e.g. citations, webpage hyperlinks)



Citation networks and Maps of science
[Börner et al., 2012]

Graph Data: Traffic Networks

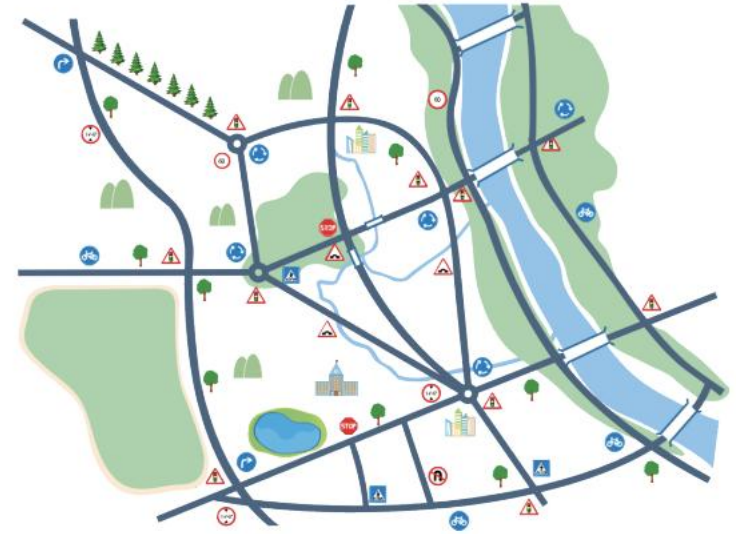
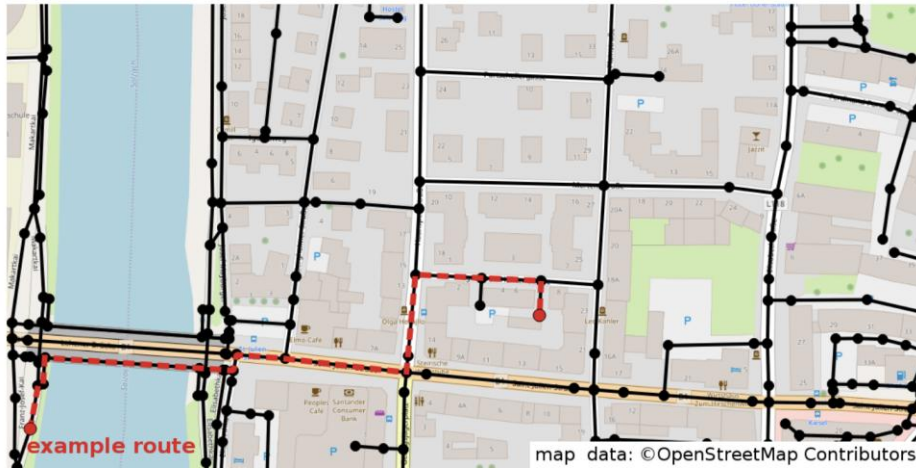


Figure 1: How can we predict how risky each road intersection is, based on nearby road features?

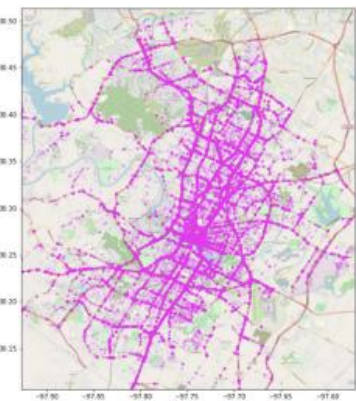


Figure 3: Traffic accident locations of Houston, Charlotte, Dallas, and Austin.

Overview: Graph Processing

Tasks

Graph Mining

PageRank

Community
Detection

Node / Graph
Similarity

...

Graph Learning

Node / Graph
Prediction

...

Systems

Small Graph Processing



NetworkX



iGraph

Large Graph Processing



APACHE
GIRAPH



GraphX

Graph Databases

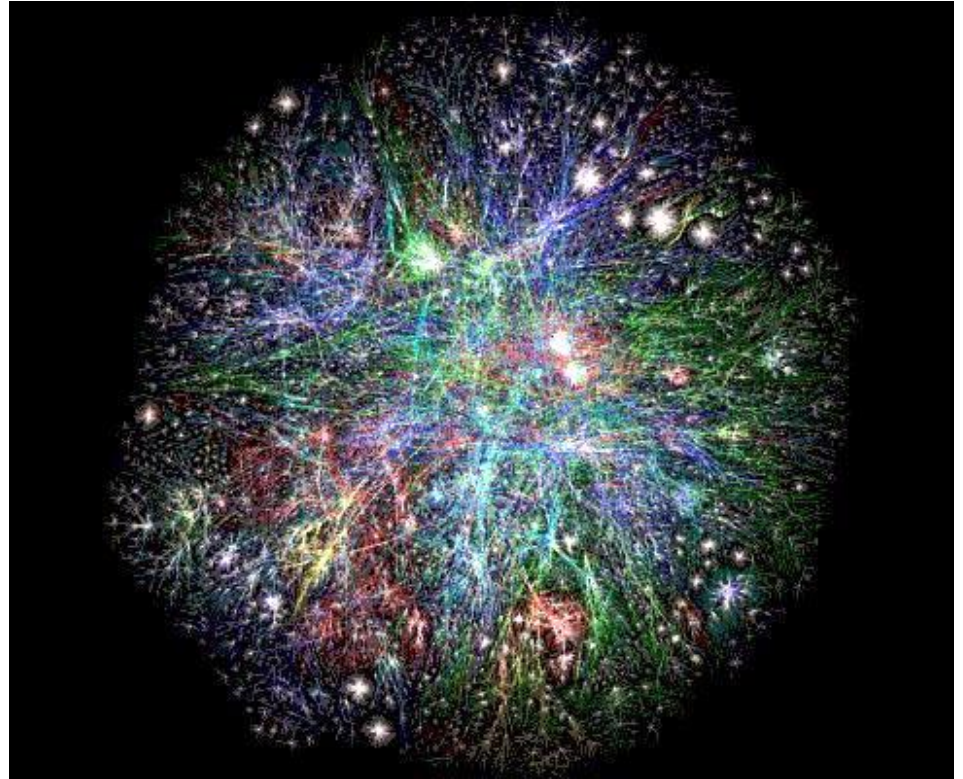


Today's Plan

- Graphs: Introduction
- **Simplified PageRank**
 - **Flow Formulation**
 - Random Walk Formulation
- PageRank (with Teleports)
- Topic Sensitive PageRank
- PageRank Implementation

Web as a Graph

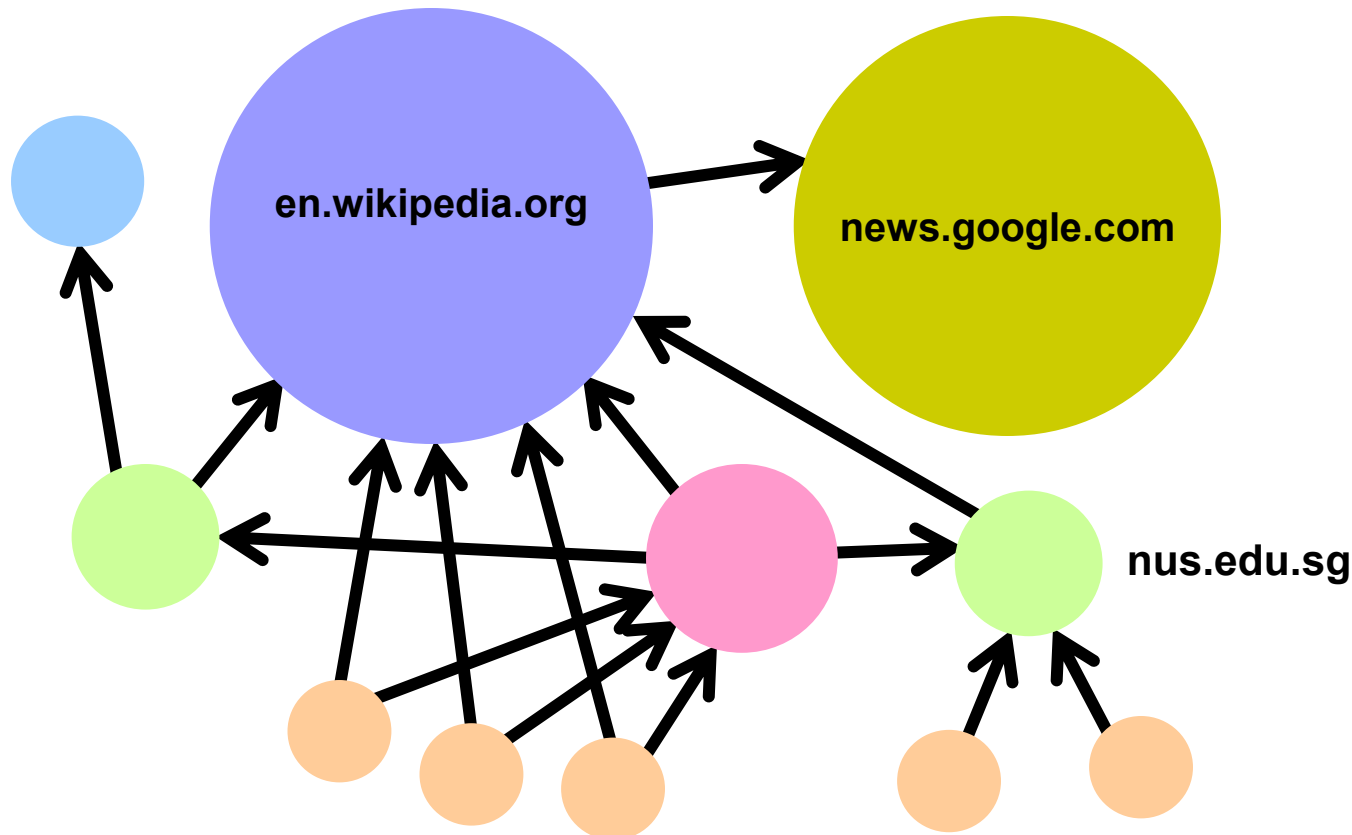
- **Web as a directed graph:**
 - **Nodes: Webpages**
 - **Edges: Hyperlinks**



Graph of the World Wide Web (<http://www.bordalierinstitute.com/>)
(The Bordalier Institute))

PageRank: Ranking Pages on the Web

- All web pages are not equally “important”: www.joe-schmoe.com vs. en.wikipedia.org
 - Measuring the **importance** of pages is necessary for many web-related tasks (e.g. search, recommendation)
 - PageRank-like methods are also used in many other applications (bioinformatics, etc.)

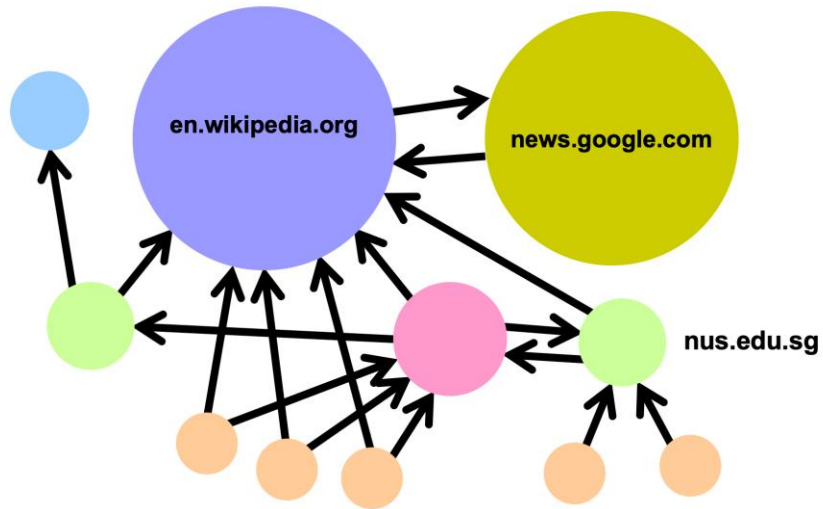


Links as Votes

- **Idea: Links as votes**

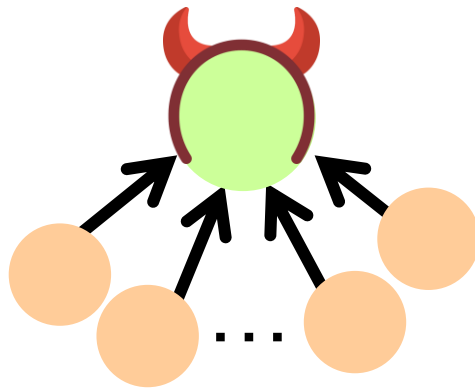
- Page is more important if it has **more in-links**
 - Assume that incoming links are harder to manipulate. For example, anyone can create an out-link from their page to en.wikipedia.edu, but it is hard to get en.wikipedia.edu to create a link to their page

- **Think of in-links as votes**



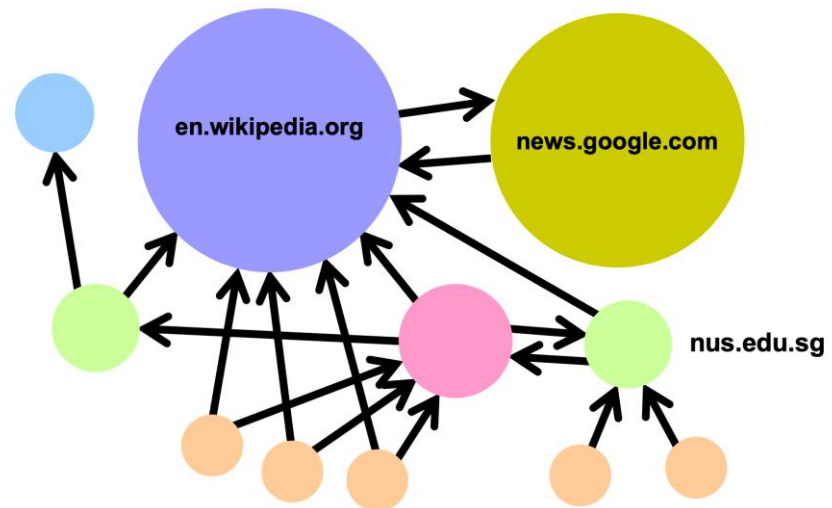
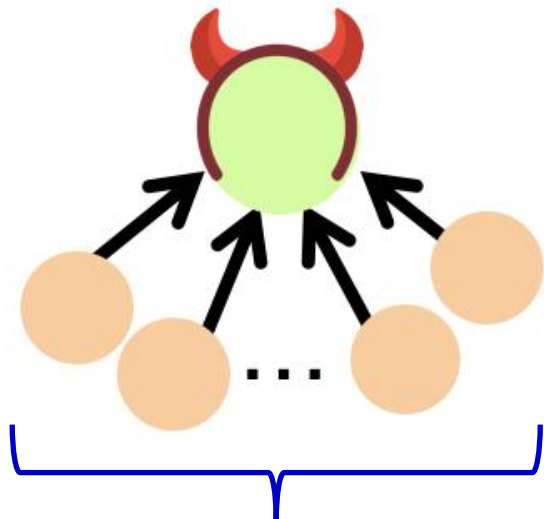
Links as Votes

- Think of in-links as votes
- Naïve solution: What if we rank each page based on its *number* of in-links?
- **Problem:** malicious user can create a huge number of 'dummy' web pages, to link to their one page, to drive up its rank!



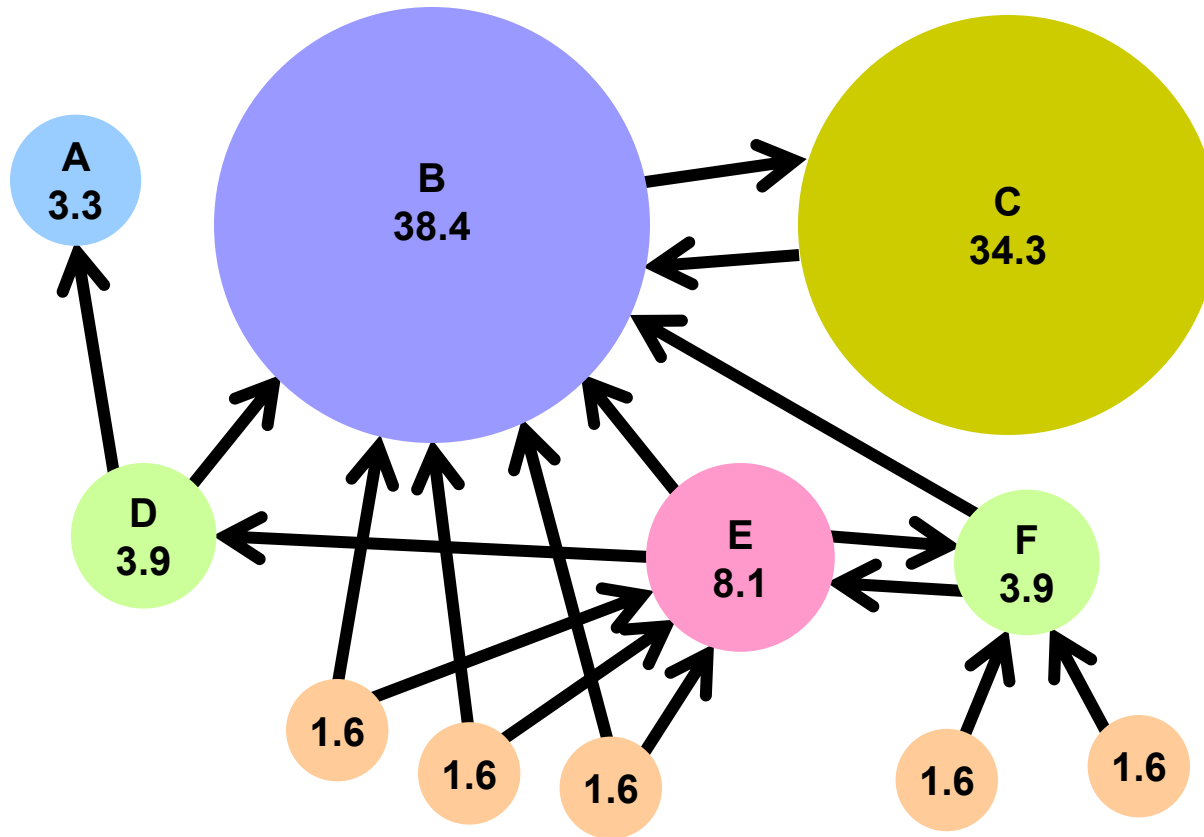
Links as Votes

- **Solution:** make the number of ‘votes’ that a page has proportional to its own importance. Then, as long as the ‘dummy’ pages themselves have low importance, they will contribute little votes as well.
- Links from important pages count more – recursive definition!
- This is the main idea of PageRank, which recursively defines the importance of a page based on the importance of the pages linking to it



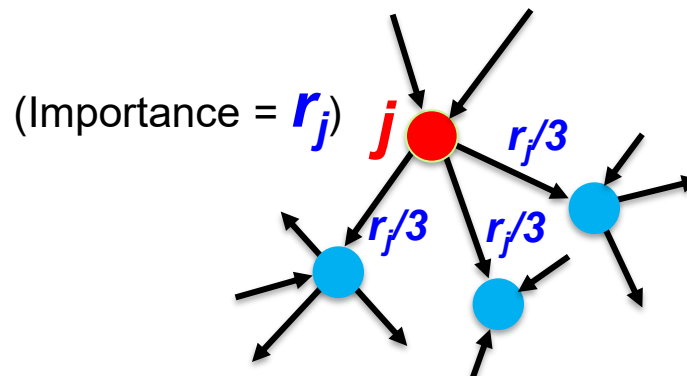
Low importance => low votes

Example: PageRank Scores



‘Voting’ Formulation

- Each link’s vote is proportional to the **importance** of its source page
- For each page j , define its “importance” (or rank) as r_j
- If page j with importance r_j has n out-links, each link gets r_j / n votes

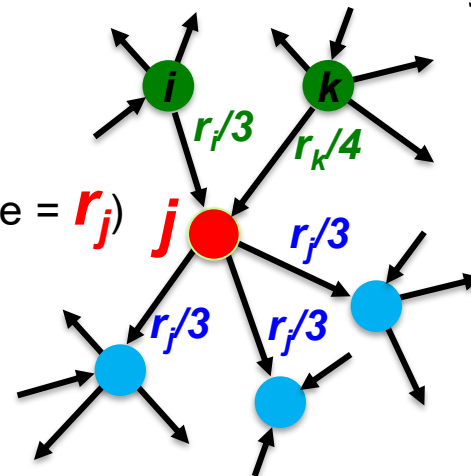


‘Voting’ Formulation

- Each link’s vote is proportional to the **importance** of its source page
- For each page j , define its “importance” (or rank) as r_j
- If page j with importance r_j has n out-links, each link gets r_j / n votes
- Page j ’s own importance is the sum of the votes on its in-links
 - Analogy: each page receives a certain amount of candies from its incoming neighbors. It distributes these candies evenly to its outgoing neighbors.

$$r_j = r_i/3 + r_k/4$$

(Importance = r_j)

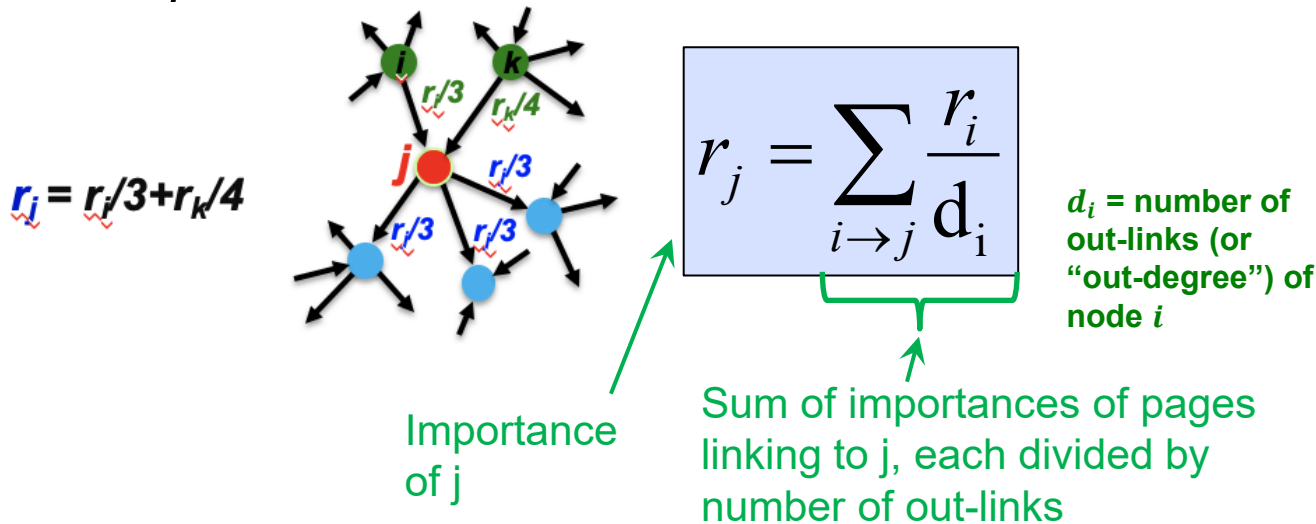


Importance of node j

Importance node j receives from its incoming edges

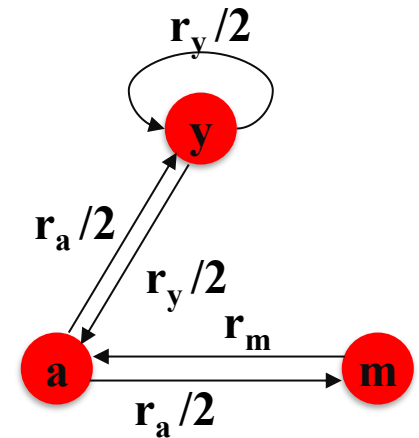
PageRank: The “Flow” Model

- A “vote” from an important page is worth more
- A page is important if it is pointed to by other important pages
- Define a “rank” or importance r_j for page j



“Simplified PageRank”

The web in 1839



“Flow” equations:

$$r_y = r_y/2 + r_a/2$$

$$r_a = r_y/2 + r_m$$

$$r_m = r_a/2$$

Solving the Flow Equations

- 3 equations, 3 unknowns, 1 redundant equation

- No unique solution
- All solutions are rescalings of each other

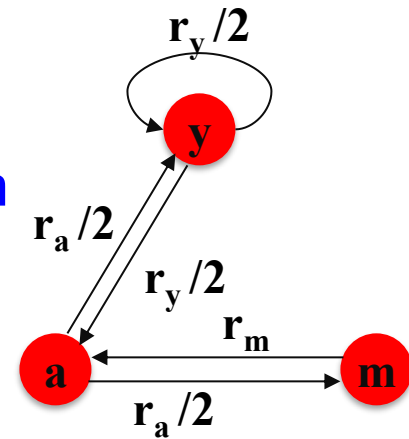
- Additional constraint forces uniqueness:

- $r_y + r_a + r_m = 1$
- Solution** (via substitution, calculations are skipped):

$$r_y = \frac{2}{5}, \quad r_a = \frac{2}{5}, \quad r_m = \frac{1}{5}$$

- Solving the equations through substitution or 'Gaussian elimination' works for small examples, but we need a better method for large web-size graphs

- We need a new formulation!**



Flow equations:

$$r_y = r_y/2 + r_a/2$$

$$r_a = r_y/2 + r_m$$

$$r_m = r_a/2$$

PageRank: Matrix Formulation

- **Stochastic adjacency matrix M**

- Let page i has d_i out-links
- If $i \rightarrow j$, then $M_{ji} = \frac{1}{d_i}$ else $M_{ji} = 0$
 - M is a **column stochastic matrix**
 - Columns sum to 1

- **Rank vector r** : vector with an entry per page

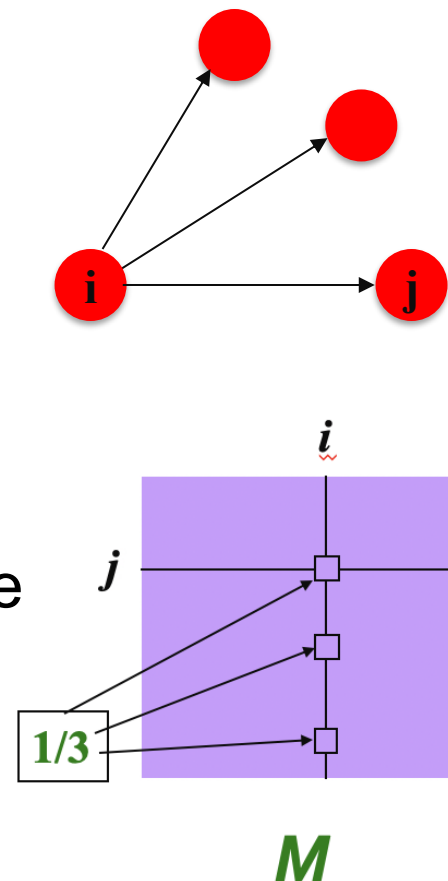
- r_i is the importance score of page i
- $\sum_i r_i = 1$

- **The flow equations can be written**

$$r = M \cdot r$$

This is equivalent to our earlier formulation:

$$\text{For all nodes } j: r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$



Why are the 2 formulations equivalent?

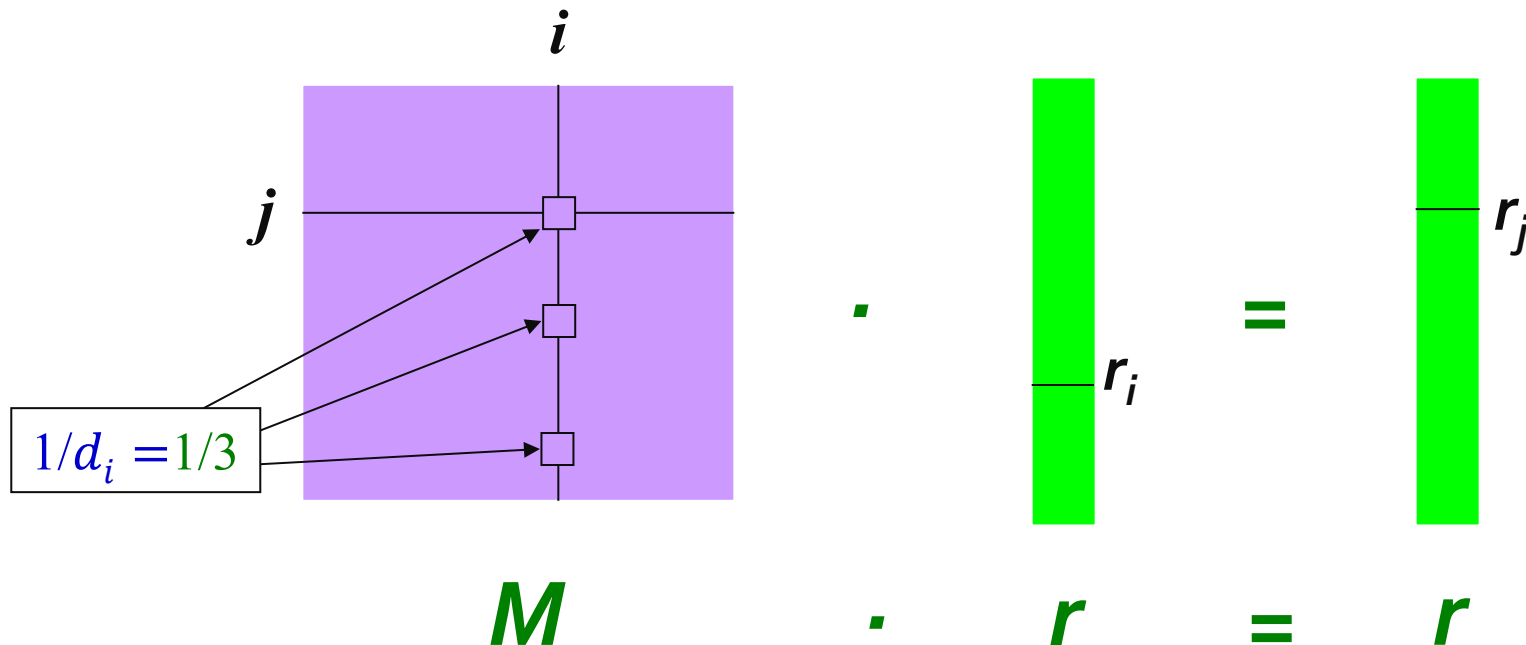
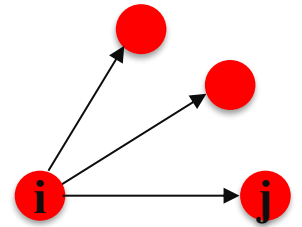
- Remember the flow equation:

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$

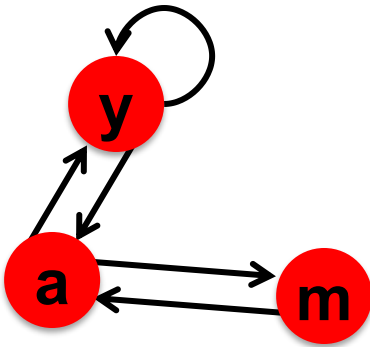
- Flow equation in the matrix form

$$M \cdot r = r$$

- Suppose page i links to 3 pages, including j



Example: Flow Equations & M



	y	a	m
y	$\frac{1}{2}$	$\frac{1}{2}$	0
a	$\frac{1}{2}$	0	1
m	0	$\frac{1}{2}$	0

$$r = M \cdot r$$

$$r_y = r_y / 2 + r_a / 2$$

$$r_a = r_y / 2 + r_m$$

$$r_m = r_a / 2$$

$$\begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & 1 \\ 0 & \frac{1}{2} & 0 \end{bmatrix} \begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix}$$

Solving the Flow Equations

- Given the flow equation:

$$r = M \cdot r$$

- We can efficiently solve for r !
The method is called Power iteration

Power Iteration Method

- Given a web graph with n nodes, where the nodes are pages and edges are hyperlinks

Power iteration: a simple iterative scheme

- Suppose there are N web pages
- Initialize: $\mathbf{r}^{(0)} = [1/N, \dots, 1/N]^T$
- Iterate: $\mathbf{r}^{(t+1)} = \mathbf{M} \cdot \mathbf{r}^{(t)}$
- Stop when $|\mathbf{r}^{(t+1)} - \mathbf{r}^{(t)}|_1 < \varepsilon$

Equivalent!

$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

d_i out-degree of node i

$|\mathbf{x}|_1 = \sum_{1 \leq i \leq N} |x_i|$ is the L_1 norm

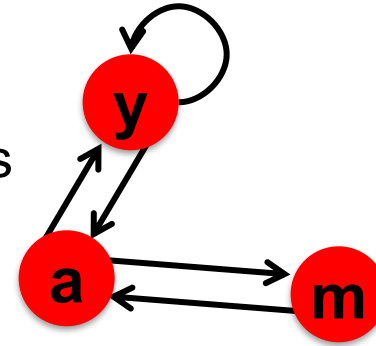
Can use any other vector norm, e.g., Euclidean

- Intuitive interpretation of power iteration:** each node starts with equal importance (of $1/N$). During each step, each node passes its current importance along its outgoing edges, to its neighbors.

Power Iteration: Example

Power Iteration:

- Suppose there are N web pages
- Initialize: $\mathbf{r}^{(0)} = [1/N, \dots, 1/N]^T$
- Iterate: $\mathbf{r}^{(t+1)} = \mathbf{M} \cdot \mathbf{r}^{(t)}$
- Stop when $|\mathbf{r}^{(t+1)} - \mathbf{r}^{(t)}|_1 < \varepsilon$



$$\begin{aligned} \mathbf{r}_y &= \mathbf{r}_y/2 + \mathbf{r}_a/2 \\ \mathbf{r}_a &= \mathbf{r}_y/2 + \mathbf{r}_m \\ \mathbf{r}_m &= \mathbf{r}_a/2 \end{aligned}$$

○ Example:

$$\begin{bmatrix} \mathbf{r}_y \\ \mathbf{r}_a \\ \mathbf{r}_m \end{bmatrix} = \begin{array}{ccccc} 1/3 & 1/3 & 5/12 & 9/24 & 2/5 \\ 1/3 & 3/6 & 1/3 & 11/24 & \dots & 2/5 \\ 1/3 & 1/6 & 3/12 & 1/6 & & 1/5 \end{array}$$

Iteration 0 Iteration 1 ...

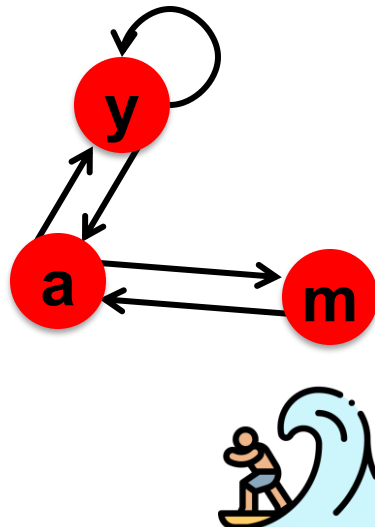
Today's Plan

- Graphs: Introduction
- **Simplified PageRank**
 - Flow Formulation
 - **Random Walk Formulation**
- PageRank (with Teleports)
- Topic Sensitive PageRank
- PageRank Implementation

Random Walk Interpretation

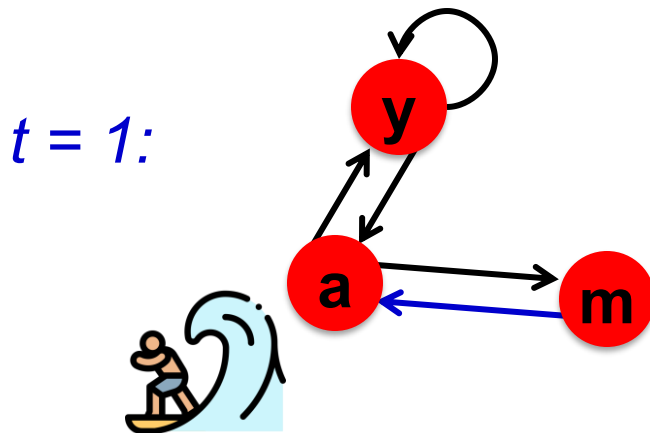
- Imagine a random web surfer:
 - At time $t = 0$, surfer starts on a random page
 - At any time t , surfer is on some page i
 - At time $t + 1$, the surfer follows an out-link from i uniformly at random
 - Process repeats indefinitely

$t = 0$:



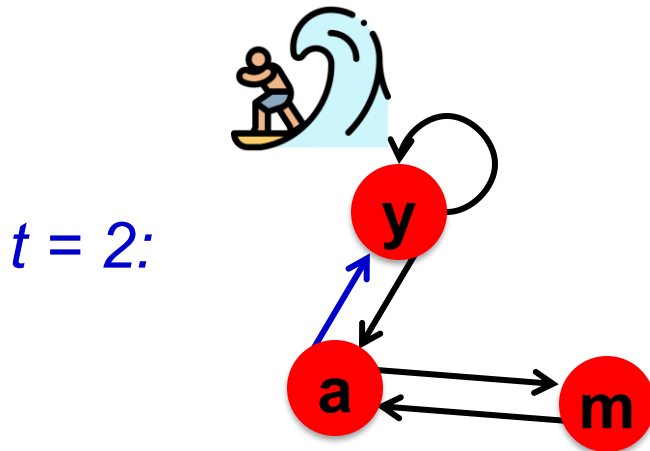
Random Walk Interpretation

- Imagine a random web surfer:
 - At time $t = 0$, surfer starts on a random page
 - At any time t , surfer is on some page i
 - At time $t + 1$, the surfer follows an out-link from i uniformly at random
 - Process repeats indefinitely



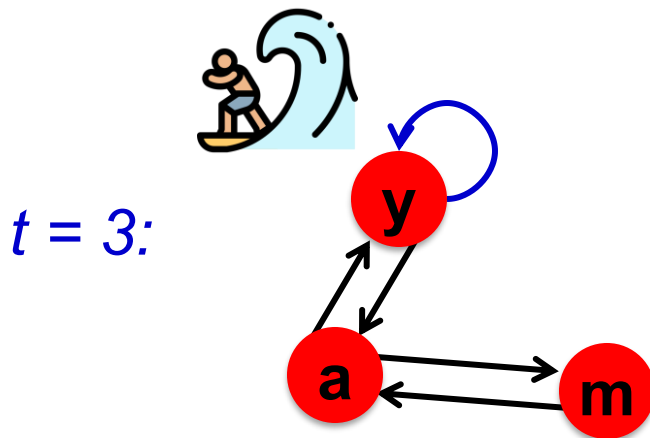
Random Walk Interpretation

- Imagine a random web surfer:
 - At time $t = 0$, surfer starts on a random page
 - At any time t , surfer is on some page i
 - At time $t + 1$, the surfer follows an out-link from i uniformly at random
 - Process repeats indefinitely



Random Walk Interpretation

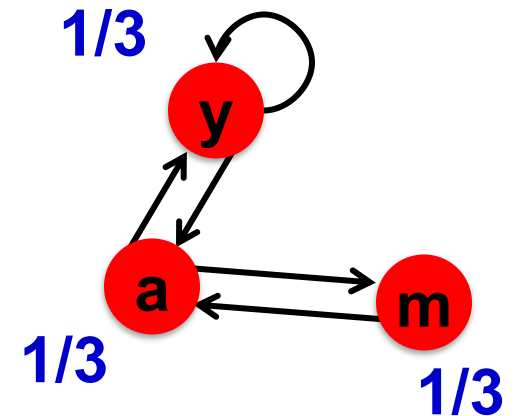
- Imagine a random web surfer:
 - At time $t = 0$, surfer starts on a random page
 - At any time t , surfer is on some page i
 - At time $t + 1$, the surfer follows an out-link from i uniformly at random
 - Process repeats indefinitely



Random Walk Interpretation

- Imagine a random web surfer:
 - At time $t = 0$, surfer starts on a random page
 - At any time t , surfer is on some page i
 - At time $t + 1$, the surfer follows an out-link from i uniformly at random
 - Process repeats indefinitely
- Let:
 - $p(t)$... vector whose i th coordinate is the prob. that the surfer is at page i at time t
 - So, $p(t)$ is a probability distribution over pages

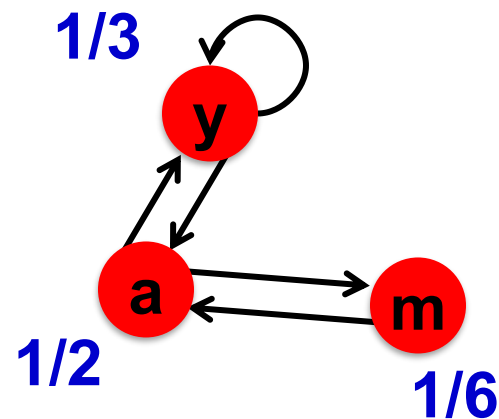
$p(t)$ where $t = 0$:



Random Walk Interpretation

- Imagine a random web surfer:
 - At time $t = 0$, surfer starts on a random page
 - At any time t , surfer is on some page i
 - At time $t + 1$, the surfer follows an out-link from i uniformly at random
 - Process repeats indefinitely
- Let:
 - $p(t)$... vector whose i th coordinate is the prob. that the surfer is at page i at time t
 - So, $p(t)$ is a probability distribution over pages

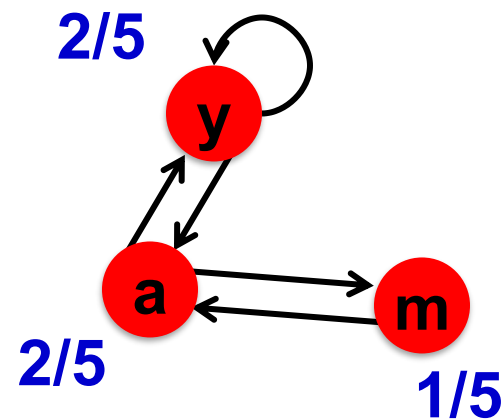
$p(t)$ where $t = 1$:



Random Walk Interpretation

- Imagine a random web surfer:
 - At time $t = 0$, surfer starts on a random page
 - At any time t , surfer is on some page i
 - At time $t + 1$, the surfer follows an out-link from i uniformly at random
 - Process repeats indefinitely
- Let:
 - $\mathbf{p}(t)$... vector whose i th coordinate is the prob. that the surfer is at page i at time t
 - So, $\mathbf{p}(t)$ is a probability distribution over pages
- **Stationary Distribution:** as $t \rightarrow \infty$, the probability distribution approaches a 'steady state' representing the long term probability that the random walker is at each node, which are the PageRank scores

$\mathbf{p}(t)$ where $t \rightarrow \infty$:



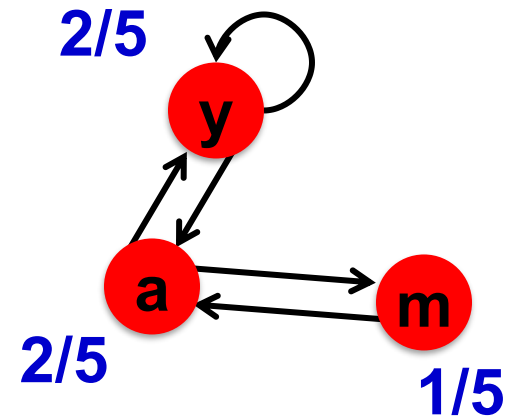
Equivalence between Random Walk and Flow Formulations

- Where is the surfer at time $t+1$?
 - Follows a link uniformly at random

$$p(t + 1) = M \cdot p(t)$$

- Suppose the random walk reaches a stationary state p_s : then $p_s = M \cdot p_s$
- In the previous 'flow' formulation of PageRank, the rank vector r was also defined by $r = M \cdot r$
 - Thus, the two (recursive) definitions are the same!

$p(t)$ where $t \rightarrow \infty$:



Today's Plan

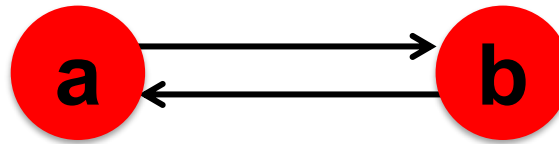
- Graphs: Introduction
- Simplified PageRank
 - Flow Formulation
 - Random Walk Formulation
- **PageRank (with Teleports)**
- Topic Sensitive PageRank
- PageRank Implementation

PageRank: Three Questions

$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i} \quad \text{or equivalently} \quad r = Mr$$

- Does this converge?
- Does it converge to what we want?
- Are results reasonable?

Does this converge?



$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

○ Answer: not always. Example:

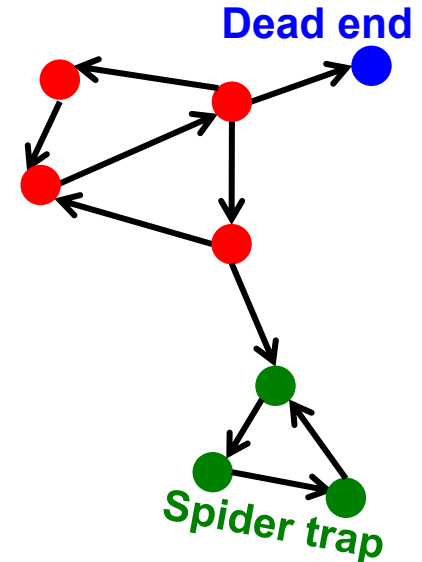
r_a	=	1	0	1	0
r_b		0	1	0	1

Iteration 0, 1, 2, ...

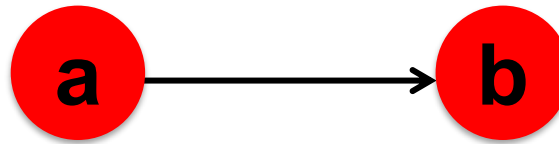
Does it converge to what we want?

Answer: not always. 2 problems:

- (1) Some pages are **dead ends** (have no out-links)
 - Random walk has “nowhere” to go to
 - Such pages cause importance to “leak out”
- (2) **Spider traps:** (all out-links are within the group)
 - Random walk gets “stuck” in a trap
 - And eventually spider traps absorb all importance
 - One node spider trap: the node has one out-link pointing to itself
 - Multiple nodes spider trap: needs to be a proper connected subset of the graph



Problem 1: Dead Ends



$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

○ Example:

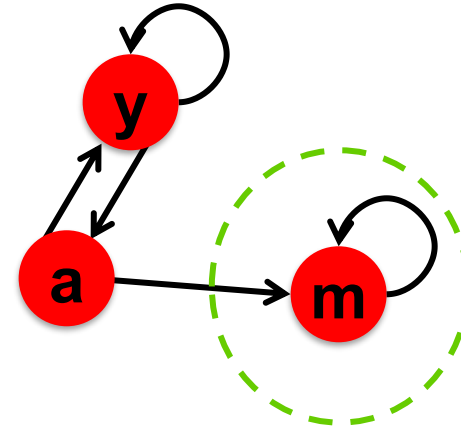
$$\begin{array}{c} r_a \\ r_b \end{array} = \begin{array}{cccc} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{array}$$

Iteration 0, 1, 2, ...

Problem 2: Spider Traps

Power Iteration:

- Set $r_j = 1/N$
- $r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$
 - And iterate



Example:

$$\begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix} = \begin{array}{cccccc} 1/3 & 2/6 & 3/12 & 5/24 & & 0 \\ 1/3 & 1/6 & 2/12 & 3/24 & \dots & 0 \\ 1/3 & 3/6 & 7/12 & 16/24 & & 1 \end{array}$$

Iteration 0, 1, 2, ...

$$r_y = r_y/2 + r_a/2$$

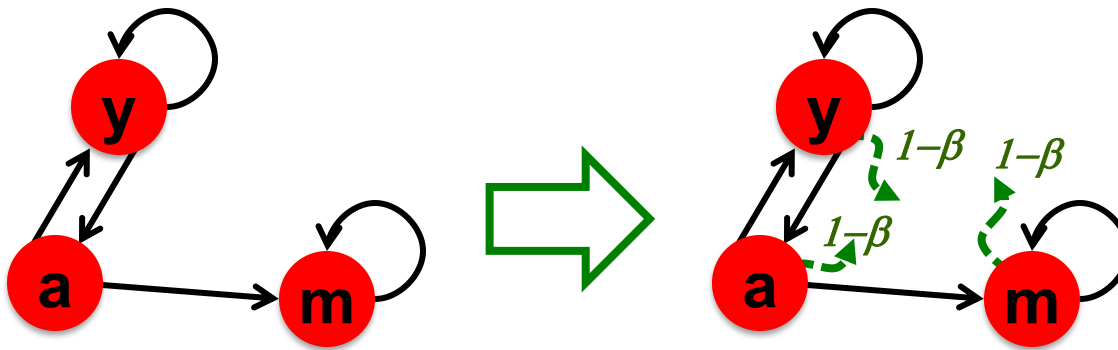
$$r_a = r_y/2$$

$$r_m = r_a/2 + r_m$$

All the PageRank score gets “trapped” in node m.

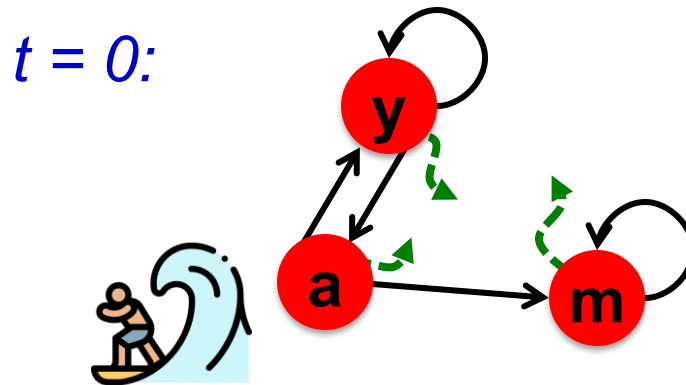
Solution: Teleports!

- The Google solution for spider traps: **At each time step, the random surfer has two options**
 - With prob. β , follow a link at random
 - With prob. $1-\beta$, jump to some random page
 - Common values for β are in the range 0.8 to 0.9



Solution: Teleports!

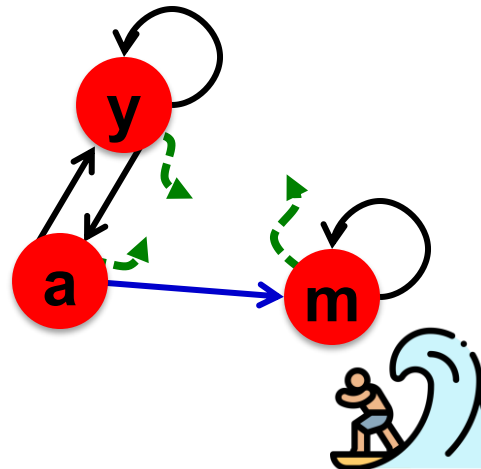
- The Google solution for spider traps: **At each time step, the random surfer has two options**
 - With prob. β , follow a link at random
 - With prob. $1-\beta$, jump to some random page
 - Common values for β are in the range 0.8 to 0.9



Solution: Teleports!

- The Google solution for spider traps: **At each time step, the random surfer has two options**
 - With prob. β , follow a link at random
 - With prob. $1-\beta$, jump to some random page
 - Common values for β are in the range 0.8 to 0.9

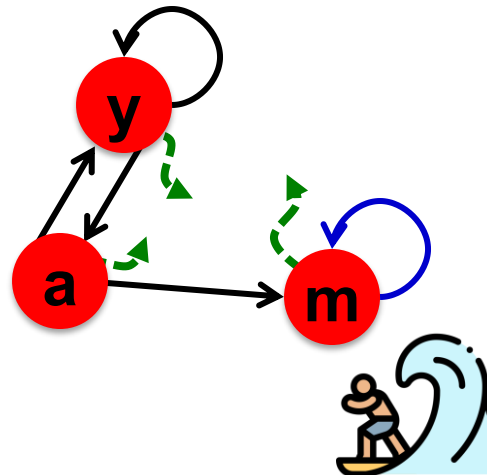
$t = 1$:



Solution: Teleports!

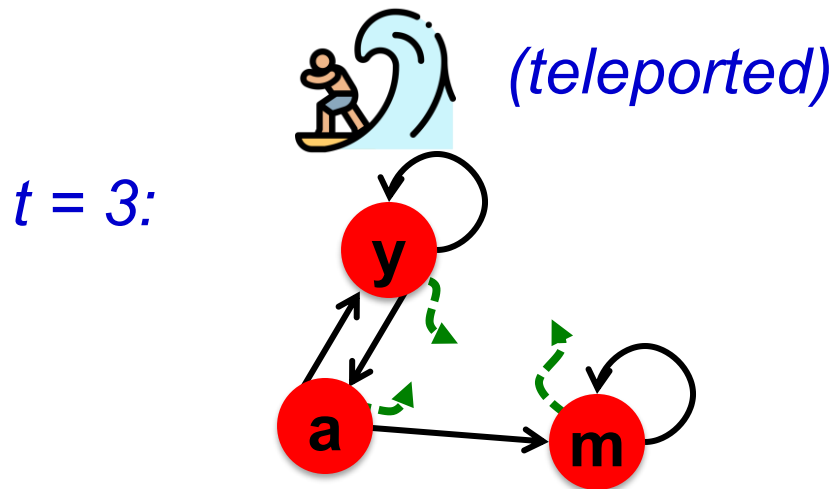
- The Google solution for spider traps: **At each time step, the random surfer has two options**
 - With prob. β , follow a link at random
 - With prob. $1-\beta$, jump to some random page
 - Common values for β are in the range 0.8 to 0.9

$t = 2:$



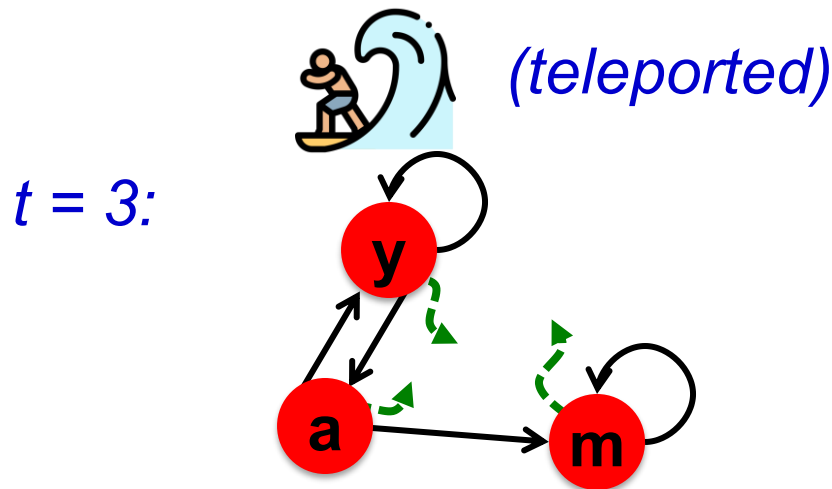
Solution: Teleports!

- The Google solution for spider traps: **At each time step, the random surfer has two options**
 - With prob. β , follow a link at random
 - With prob. $1-\beta$, jump to some random page
 - Common values for β are in the range 0.8 to 0.9



Solution: Teleports!

- The Google solution for spider traps: **At each time step, the random surfer has two options**
 - With prob. β , follow a link at random
 - With prob. $1-\beta$, jump to some random page
 - Common values for β are in the range 0.8 to 0.9

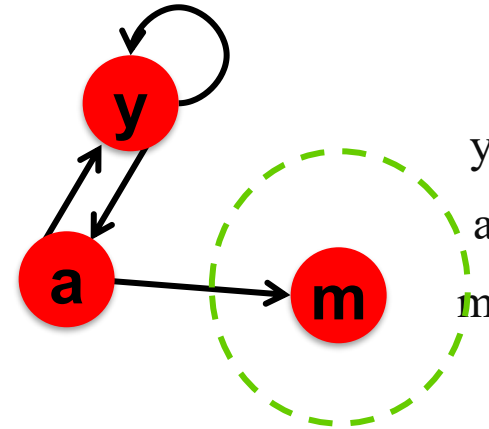


- **Conclusion:** surfer will quickly teleport out of any spider trap

Problem: Dead Ends

Power Iteration:

- Set $r_j = 1/N$
- $r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$
 - And iterate



	y	a	m
y	1/2	1/2	0
a	1/2	0	0
m	0	1/2	0

$$r_y = r_y/2 + r_a/2$$

$$r_a = r_y/2$$

$$r_m = r_a/2$$

Example:

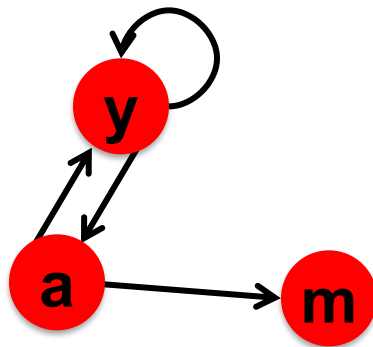
$$\begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix} = \begin{matrix} & \begin{matrix} 1/3 & 2/6 & 3/12 & 5/24 & & 0 \end{matrix} \\ \begin{matrix} 1/3 & 1/6 & 2/12 & 3/24 & \dots & 0 \end{matrix} & \\ \begin{matrix} 1/3 & 1/6 & 1/12 & 2/24 & & 0 \end{matrix} & \end{matrix}$$

Iteration 0, 1, 2, ...

Here the PageRank “leaks” out since the matrix is not stochastic.

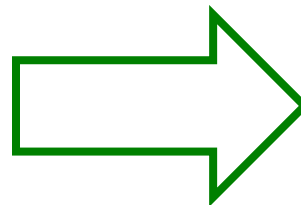
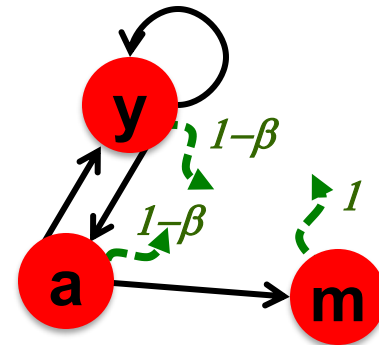
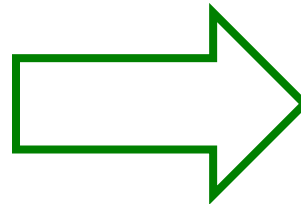
Solution: If at a Dead End, Always Teleport

- **Teleports:** Follow random teleport links with probability 1.0 from dead-ends
- Equivalently, for each dead end **m** we can preprocess the random walk matrix **M** by making **m** connected to **every node** (including itself).



M =

	y	a	m
y	$\frac{1}{2}$	$\frac{1}{2}$	0
a	$\frac{1}{2}$	0	0
m	0	$\frac{1}{2}$	0



	y	a	m
y	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{3}$
a	$\frac{1}{2}$	0	$\frac{1}{3}$
m	0	$\frac{1}{2}$	$\frac{1}{3}$

Why Teleports Solve the Problem?

Why are dead-ends and spider traps a problem and why do teleports solve the problem?

- **Spider-traps** cause random walker to get stuck in them, absorbing all importance
 - **Solution:** Never get stuck in a spider trap by teleporting out of it in a finite number of steps
- **Dead-ends** cause importance to “leak” out of the system
 - The matrix is not column stochastic
 - **Solution:** Make matrix column stochastic by always teleporting when at a dead end

Random Teleport: Equations

- At each step, random surfer has two options:
 - With probability β , follow a link at random
 - With probability $1-\beta$, jump to some random page
- **PageRank equation** [Brin-Page, 98]

$$r_j = \sum_{i \rightarrow j} \beta \underbrace{\frac{r_i}{d_i}}_{\text{Same as simplified PageRank}} + \underbrace{(1 - \beta) \frac{1}{N}}_{\text{Teleport term}}$$

d_i ... out-degree of node i

- *Compare to Simplified PageRank:*

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$

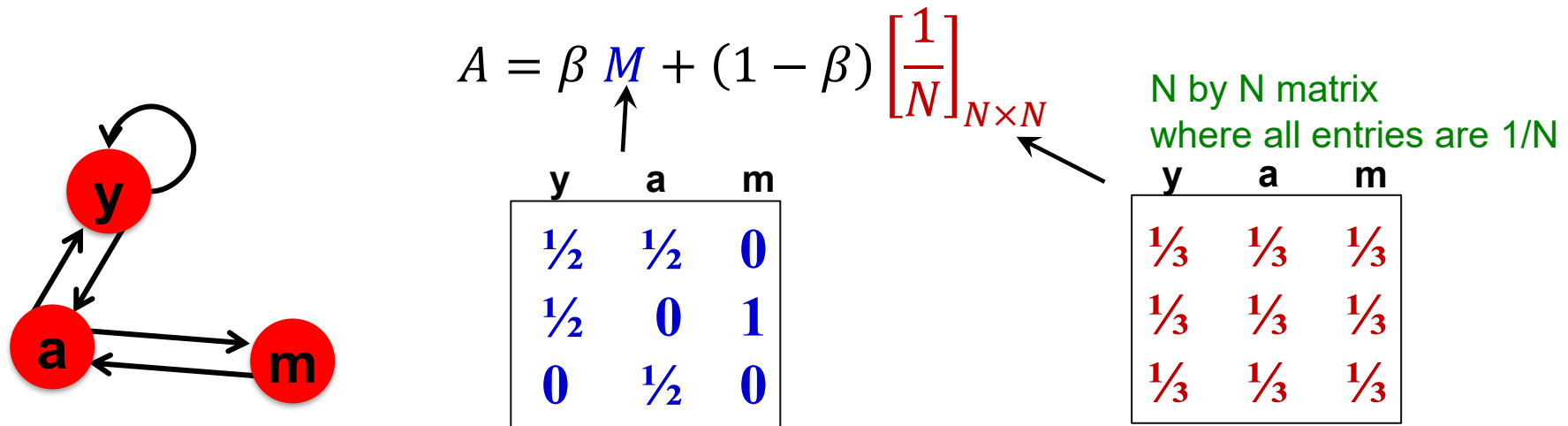
(If any dead ends exist, we assume that we have preprocessed by adding connections from them to every node, including itself)

The Google Matrix

- PageRank equation [Brin-Page, '98]

$$r_j = \sum_{i \rightarrow j} \beta \frac{r_i}{d_i} + (1 - \beta) \frac{1}{N}$$

- We can also write this as a matrix equation, by defining the **Google Matrix A**:



- PageRank equation (matrix form): $\mathbf{r} = \mathbf{A} \cdot \mathbf{r}$

- In practice $\beta = 0.8, 0.9$ (5-10 steps on average before teleport)

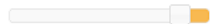
PageRank: Animation

- <https://www.learnforeverlearn.com/pagerank/>

Exploration of the Google PageRank Algorithm

Circles correspond to web pages, links correspond to hyperlinks

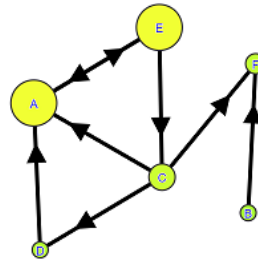
Damping Factor d:
0.85



Generate New

- Drag nodes to rearrange
- Click on node, then click on another to create link
- Double-click to add node (max 14)
- Del+mouseover deletes node
- Click on link to delete
- Mouseover link to see location in matrix

Hide Matrix Stuff



A Project by [Brag Lyon](#)
[About This Visualization](#)

Sorted
PageRank
Vector
(10 Iterations)

Explore Convergence

A	0.29
E	0.29
C	0.17
F	0.13
D	0.09
B	0.04

The PageRank vector x satisfies $x=Gx$, where

Google Matrix $G = d * [(\text{Hyperlink Matrix } H) + (\text{Dangling Nodes Matrix } A)] + \{(1-d)/N\} * (\text{NxN Matrix } U \text{ of all } 1\text{'s})$

$$G = 0.85 * \left[\begin{array}{c|ccccc} \text{Hyperlink Matrix } H & A & B & C & D & E & F \\ \hline A & & 1/3 & 1 & 1/2 & & \\ B & & & & & & \\ C & & & & 1/2 & & \\ D & & & 1/3 & & & \\ E & 1 & & & & & \\ F & & 1 & 1/3 & & & \end{array} \right] + \frac{0.15}{6} * \begin{array}{c|ccccc} \text{Matrix } U \text{ with all } 1\text{'s} & A & B & C & D & E & F \\ \hline A & 1 & 1 & 1 & 1 & 1 & 1 \\ B & 1 & 1 & 1 & 1 & 1 & 1 \\ C & 1 & 1 & 1 & 1 & 1 & 1 \\ D & 1 & 1 & 1 & 1 & 1 & 1 \\ E & 1 & 1 & 1 & 1 & 1 & 1 \\ F & 1 & 1 & 1 & 1 & 1 & 1 \end{array}$$

$$G = \begin{bmatrix} 0.03 & 0.03 & 0.31 & 0.88 & 0.45 & 0.17 \\ 0.03 & 0.03 & 0.03 & 0.03 & 0.03 & 0.17 \\ 0.03 & 0.03 & 0.03 & 0.03 & 0.45 & 0.17 \\ 0.03 & 0.03 & 0.31 & 0.03 & 0.03 & 0.17 \\ 0.88 & 0.03 & 0.03 & 0.03 & 0.03 & 0.17 \\ 0.03 & 0.88 & 0.31 & 0.03 & 0.03 & 0.17 \end{bmatrix}$$

Some Problems with Page Rank

- **Measures generic popularity of a page**
 - Doesn't consider popularity based on specific topics
 - **Solution:** Topic-Specific PageRank
- **Uses a single measure of importance**
 - Other models of importance
 - **Solution:** Hubs-and-Authorities
- **Susceptible to Link spam**
 - Artificial link topographies created in order to boost page rank
 - **Solution:** TrustRank

Today's Plan

- Graphs: Introduction
- Simplified PageRank
 - Flow Formulation
 - Random Walk Formulation
- PageRank (with Teleports)
- **Topic Specific PageRank**
- PageRank Implementation

Topic-Specific PageRank

- **Instead of generic popularity, can we measure popularity within a topic?**
- **Goal:** Evaluate Web pages not just according to their popularity, but by how close they are to a particular topic, e.g. “sports” or “history”
- **Allows search queries to be answered based on interests of the user**
 - **Example:** Query “Trojan” wants different pages depending on whether you are interested in sports, history and computer security

Topic-Specific PageRank

- Random walker has a small probability of teleporting at any step
- **Teleport can go to:**
 - **Standard PageRank:** Any page with equal probability
 - To avoid dead-end and spider-trap problems
 - **Topic Specific PageRank:** A topic-specific set of “relevant” pages (**teleport set**)
- **Idea: Bias the random walk**
 - When random walker teleports, it picks a page from a set S
 - S contains only pages that are relevant to the topic
 - For each teleport set S , we get a different vector r_S

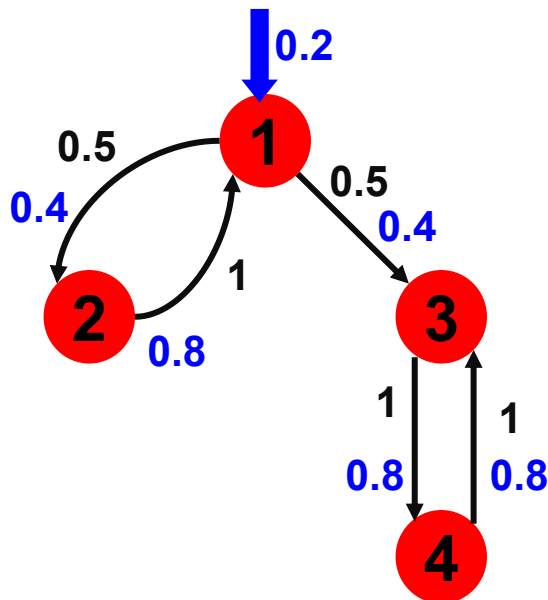
Matrix Formulation

- To make this work all we need is to update the teleportation part of the PageRank formulation:

$$A_{ij} = \begin{cases} \beta M_{ij} + (1 - \beta)/|S| & \text{if } i \in S \\ \beta M_{ij} + 0 & \text{otherwise} \end{cases}$$

- A is stochastic!
- We weighted all pages in the teleport set S equally
 - Could also assign different weights to pages!
- Compute as for regular PageRank:
 - Multiply by M , then add a vector
 - Maintains sparseness

Example: Topic-Specific PageRank



$$S = \{1\} \quad \beta = 0.8$$

Node	Iteration				
	0	1	2	...	stable
1	0.25	0.4	0.28		0.294
2	0.25	0.1	0.16		0.118
3	0.25	0.3	0.32		0.327
4	0.25	0.2	0.24		0.261

Probabilities (no teleport)

Probabilities (with teleport)

$S=\{1\}, \beta=0.9:$

$r=[0.17, 0.07, 0.40, 0.36]$

$S=\{1\}, \beta=0.8:$

$r=[0.29, 0.11, 0.32, 0.26]$

$S=\{1\}, \beta=0.7:$

$r=[0.39, 0.14, 0.27, 0.19]$

$S=\{1,2,3,4\}, \beta=0.8:$

$r=[0.13, 0.10, 0.39, 0.36]$

$S=\{1,2,3\}, \beta=0.8:$

$r=[0.17, 0.13, 0.38, 0.30]$

$S=\{1,2\}, \beta=0.8:$

$r=[0.26, 0.20, 0.29, 0.23]$

$S=\{1\}, \beta=0.8:$

$r=[0.29, 0.11, 0.32, 0.26]$

Discovering the Topic Vector S

- **Create different PageRanks for different topics**

- The 16 DMOZ top-level categories:
 - arts, business, sports,...

- **Which topic ranking to use?**

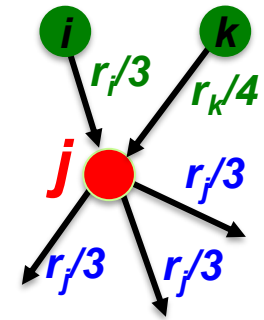
- User can pick from a menu
- Classify query into a topic
- Can use the **context** of the query
 - E.g., query is launched from a web page talking about a known topic
 - History of queries e.g., “basketball” followed by “Jordan”
- User context, e.g., user’s bookmarks, ...

Today's Plan

- Graphs: Introduction
- Simplified PageRank
 - Flow Formulation
 - Random Walk Formulation
- PageRank with Teleports
- Topic Sensitive PageRank
- PageRank Implementation

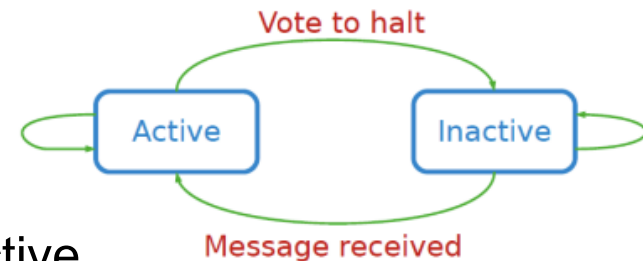
Characteristics of Graph Algorithms

- What are some common features of graph algorithms?
 - Local computations at each vertex
 - Passing messages to other vertex
- **Think like a vertex:** algorithms are implemented from the view of a single vertex, performing one iteration based on messages from its neighbor
 - Similar to MapReduce, the user only implements a simple function, `compute()`, that describes the algorithm's behavior at one vertex, in one step.
 - The framework abstracts away the scheduling / implementation details.



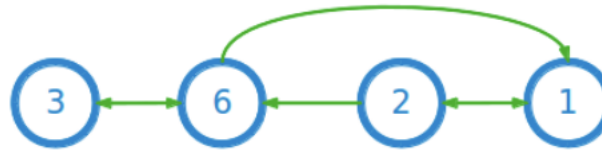
Pregel: Computational Model

- Computation consists of a series of **supersteps**
- In each superstep, the framework **invokes a user-defined function**, `compute()`, **for each vertex** (conceptually in parallel)
- `compute()` specifies **behavior at a single vertex** v and a superstep s :
 - It can **read messages** sent to v in superstep $s - 1$;
 - It can **send messages** to other vertices that will be read in superstep $s + 1$;
 - It can **read or write the value** of v and the value of its outgoing edges (or even add or remove edges)
- Termination:
 - A vertex can choose to deactivate itself
 - Is “woken up” if new messages received
 - Computation halts when all vertices are inactive



Example: Computing Max Value

- **Task:** compute the maximum over the values in the vertices in this graph:



- **Summary of approach:**
 - Each vertex repeatedly sends its current value to its neighbors (as “messages”). Then each vertex updates its value to the maximum over its own value, and all the messages it receives. This process continues until all vertex values stop changing.

Example: Computing Max Value

Compute(v, messages):

changed = False

for m in messages:

if v.getValue() < m:

v.setValue(m)

changed = True

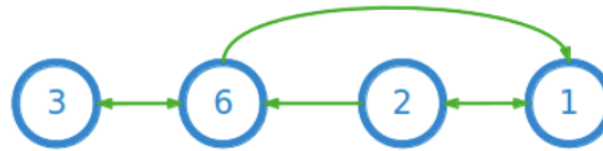
if changed:

for each outneighbor w:

sendMessage(w, v.getValue())

else:

voteToHalt()

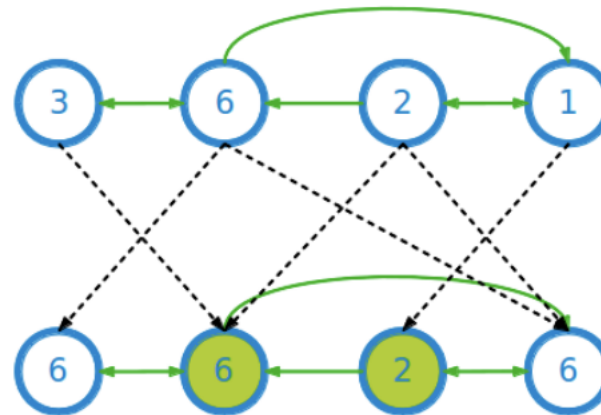


Super step 0

Example: Computing Max Value

Compute(v, messages):

```
changed = False
for m in messages:
    if v.getValue() < m:
        v.setValue(m)
        changed = True
if changed:
    for each outneighbor w:
        sendMessage(w, v.getValue())
else:
    voteToHalt()
```



Super step 0

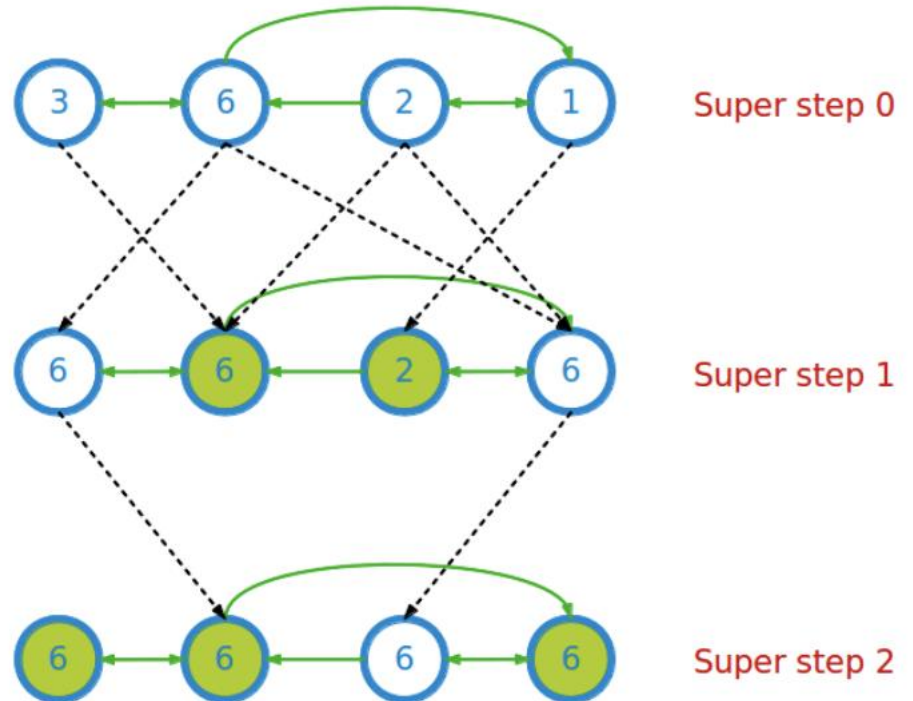
Super step 1

Voting to halt

Example: Computing Max Value

Compute(v, messages):

```
changed = False
for m in messages:
    if v.getValue() < m:
        v.setValue(m)
        changed = True
if changed:
    for each outneighbor w:
        sendMessage(w, v.getValue())
else:
    voteToHalt()
```



Example: Computing Max Value

Compute(v, messages):

changed = **False**

for m in messages:

if v.getValue() < m:

 v.setValue(m)

 changed = **True**

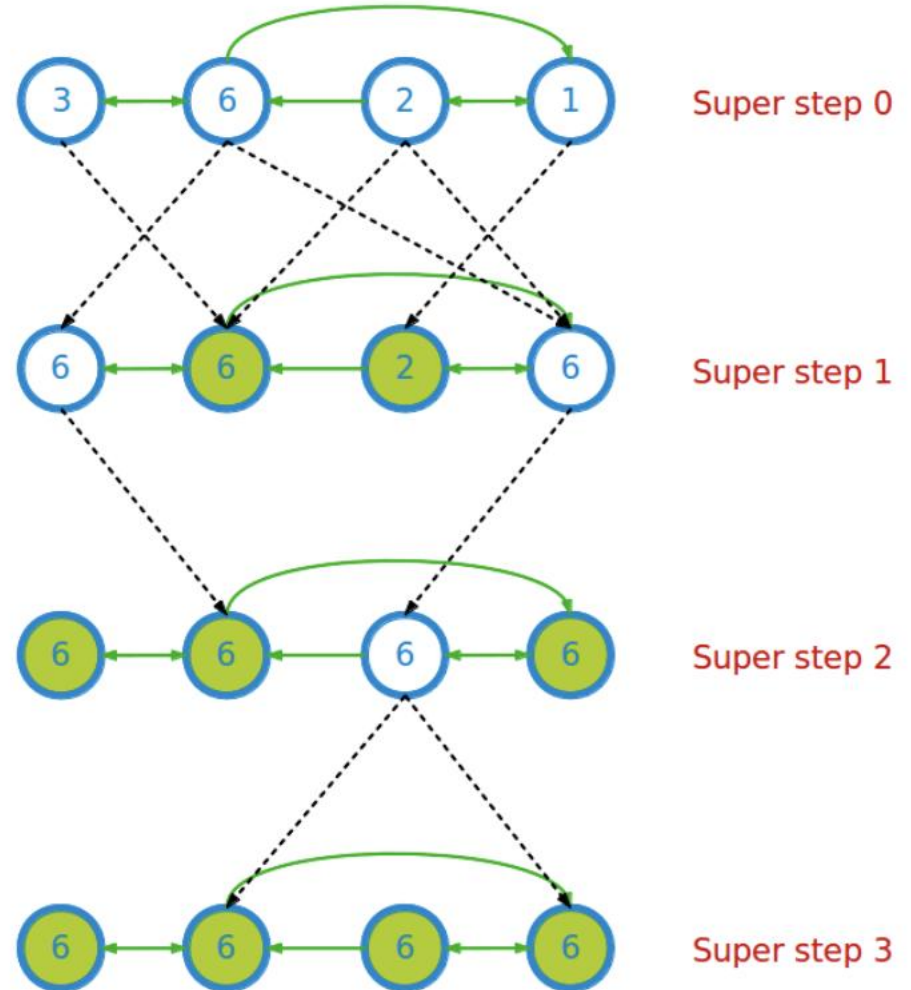
if changed:

for each outneighbor w:

 sendMessage(w, v.getValue())

else:

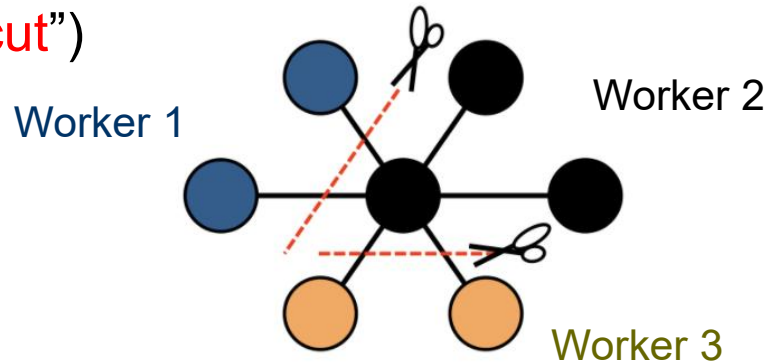
 voteToHalt()



Pregel: Implementation

- Master & workers architecture

- Vertices are hash partitioned (by default) and assigned to workers (“**edge cut**”)



- Each worker maintains the state of its portion of the graph **in memory**
- Computations happen in memory
- In each superstep, each worker loops through its vertices and executes `compute()`
- Messages from vertices are sent, either to vertices on the same worker, or to vertices on different workers (**buffered locally** and sent as a batch to reduce network traffic)

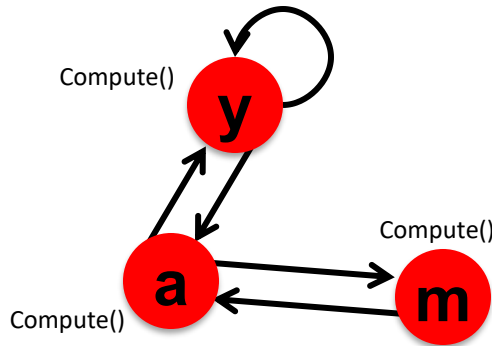
Pregel: Implementation

- Fault tolerance
 - Checkpointing to persistent storage
 - Failure detected through heartbeats
 - Corrupt workers are reassigned and reloaded from checkpoints

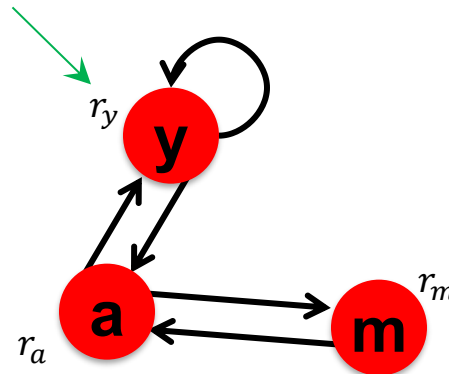
PageRank in Pregel

Node's current values, computed by summing messages from the previous Superstep and computing the PageRank update equation below.

Outgoing messages are computed by dividing each node's value equally among its outgoing neighbours (thus obtaining $\frac{r_i}{d_i}$)

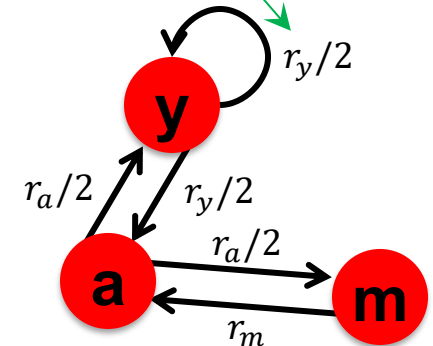


Superstep 1:
Invoke Compute()



Superstep 1:
Aggregate messages from
neighbors to compute
PageRank update

$$r_j = \sum_{i \rightarrow j} \beta \frac{r_i}{d_i} + (1 - \beta) \frac{1}{N}$$



Superstep 1:
Send message to
outgoing neighbors



Repeat this execution
till Superstep reaches
30

PageRank in Pregel

```
class PageRankVertex : public Vertex<double, void, double> {
public:
    virtual void Compute(Messagelatorator* msgs) {
        if (superstep() >= 1) {
            double sum = 0;
            for (; !msgs->Done(); msgs->Next())
                sum += msgs->Value();
            *MutableValue() = 0.15 / NumVertices() + 0.85 * sum;
        }

        if (superstep() < 30) {
            const int64 n = GetOutEdgelterator().size();
            SendMessageToAllNeighbors(GetValue() / n);
        } else {
            VoteToHalt();
        }
    }
};
```

Compute sum of
incoming messages

PageRank
update

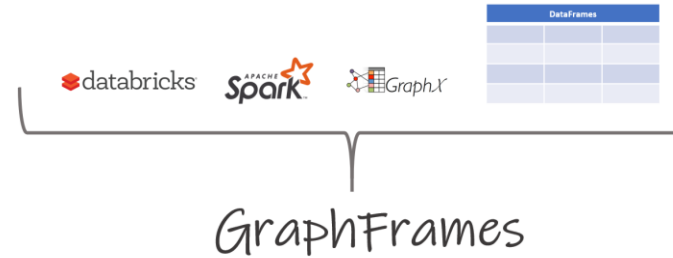
$$r_j = \sum_{i \rightarrow j} \beta \frac{r_i}{d_i} + (1 - \beta) \frac{1}{N}$$

Send outgoing messages

Stop after fixed no. of
iterations

Note: this algorithm implicitly assumes that the nodes' values (*MutableValue()) have been correctly initialized based on the initialization scheme that the user wants. In practice, you would generally do the initialization during superstep 0.

Spark for Graphs

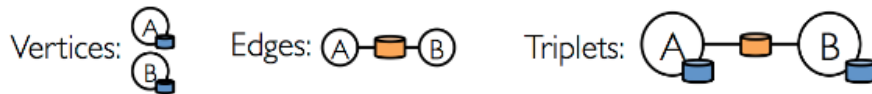


- Integration of record-oriented and graph-oriented processing
- GraphX: Extends RDDs to Resilient Distributed Property Graphs
- Property Graphs:
 - Present different views of the graph (vertices, edges, triplets)
 - Support distributed Pregel-like computation
- Graphframe: Extends GraphX to provide a DataFrame API and support for Spark's different language bindings

Spark Implementation of Page Rank

Stage 1: Join

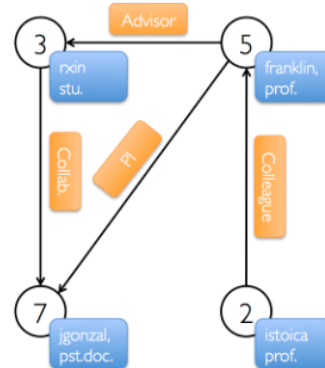
```
CREATE VIEW triplets AS
SELECT s.Id, d.Id, s.P, e.P, d.P
FROM edges AS e
JOIN vertices AS s JOIN vertices AS d
ON e.srcId = s.Id AND e.dstId = d.Id
```



Stage 2: Group-By

```
SELECT t.dstId, 0.15+0.85*sum(t.srcP*t.eP)
FROM triplets AS t GROUP BY t.dstId
```

Property Graph



Vertex Table

Id	Property (V)
3	(rxin, student)
7	(jgonzal, postdoc)
5	(franklin, professor)
2	(istoica, professor)

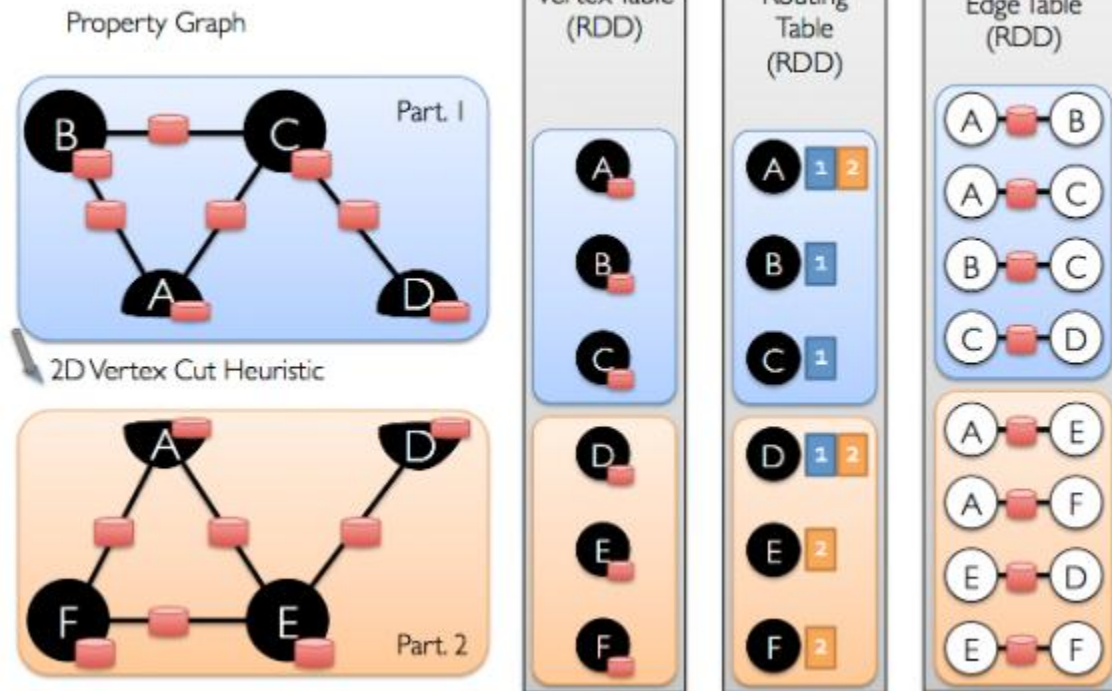
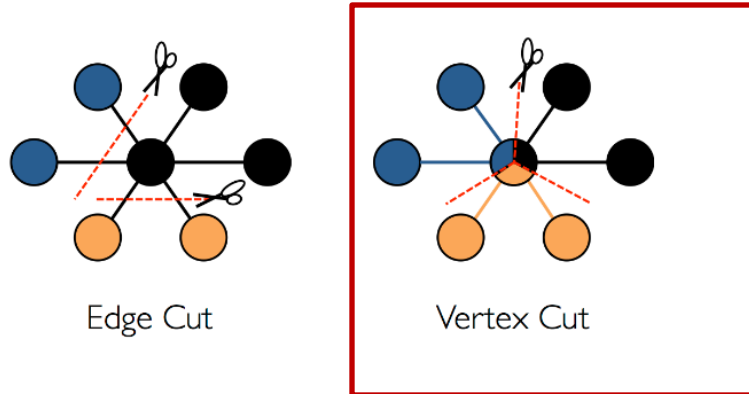
Edge Table

SrcId	DstId	Property (E)
3	7	Collaborator
5	3	Advisor
2	5	Colleague
5	7	PI

$$r_j = \sum_{i \rightarrow j} \beta \frac{r_i}{d_i} + (1 - \beta) \frac{1}{N}$$

<https://spark.apache.org/docs/latest/graphx-programming-guide.html>

Spark uses Vertex Cut to partition the graph



Practical 4: Graph Analytics using Spark Graphframes

Practical 4: Graph Analytics

Load Data

```
1 bikeStations = spark.read.option("header", "true")\
2   .csv("/databricks-datasets/definitive-guide/data/bike-data/201508_station_data.csv")
3 tripData = spark.read.option("header", "true")\
4   .csv("/databricks-datasets/definitive-guide/data/bike-data/201508_trip_data.csv")
5
```

▶ (2) Spark Jobs

▶ bikeStations: pyspark.sql.dataframe.DataFrame = [station_id: string, name: string ... 5 more fields]

▶ tripData: pyspark.sql.dataframe.DataFrame = [Trip ID: string, Duration: string ... 9 more fields]

```
1 stationVertices = bikeStations.withColumnRenamed("name", "id").distinct()
2 tripEdges = tripData\
3   .withColumnRenamed("Start Station", "src")\
4   .withColumnRenamed("End Station", "dst")
5
```

▶ stationVertices: pyspark.sql.dataframe.DataFrame = [station_id: string, id: string ... 5 more fields]

▶ tripEdges: pyspark.sql.dataframe.DataFrame = [Trip ID: string, Duration: string ... 9 more fields]

You can download all the Practical files from [Canvas/Files/Practicals.zip](#)

Comparison

- Giraph
 - An open-source implementation of Pregel by Facebook
- Spark GraphX / GraphFrame
 - Extends RDDs to Resilient Distributed Property Graphs
 - Join Vertex Table and Edge Table to capture the relationships
- Neo4j
 - Graph database + Graph processing
 - SQL like interface: Cypher Query Language

Comparison

- GraphX Paper 2015
 - GraphX can achieve performance parity compared with specialized graph systems

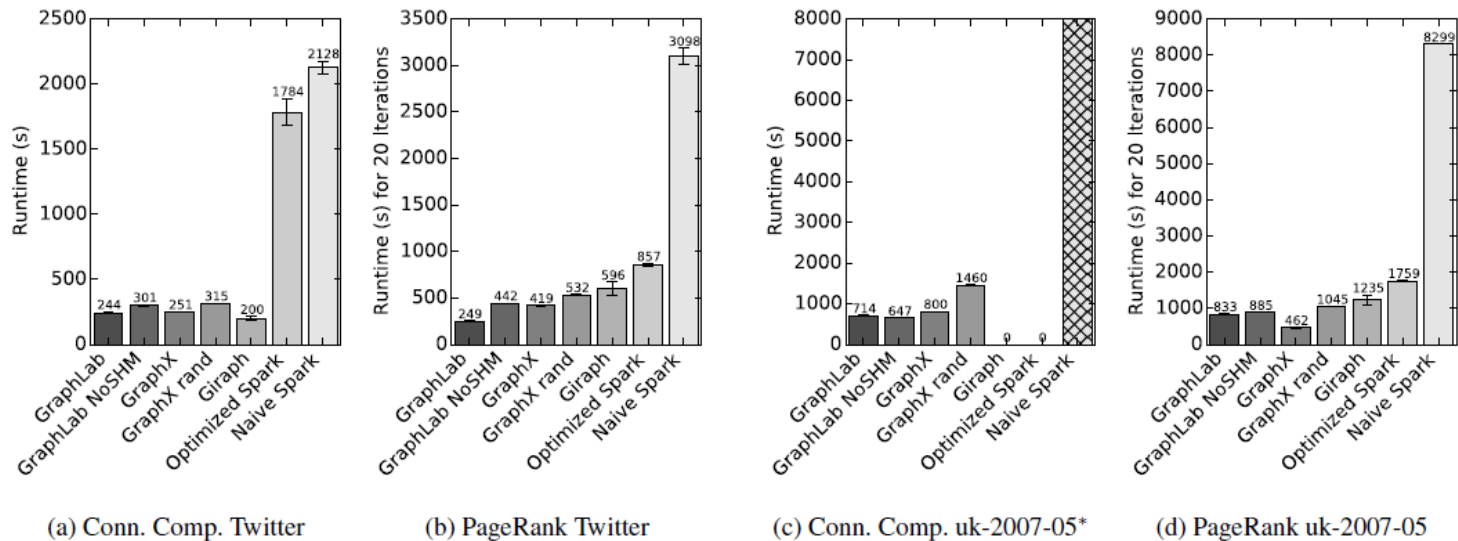


Figure 7: **System Performance Comparison.** (c) Spark did not finish within 8000 seconds, Giraph and Spark + Part. ran out of memory.

Comparison

Facebook 2016

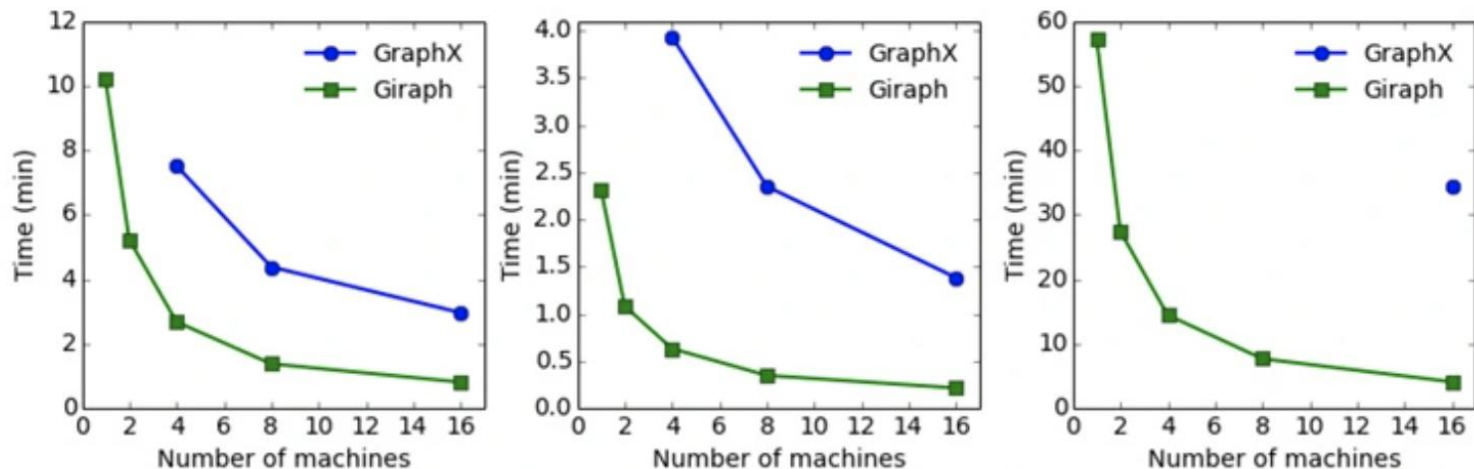
- Giraph performs better than GraphX
- Efficiency Comparison (on ~ trillion-scale Facebook social graph)

FACEBOOK Engineering



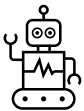
POSTED ON OCTOBER 19, 2016 TO [CORE DATA](#), [DATA INFRASTRUCTURE](#)

A comparison of state-of-the-art graph processing systems



Take-away in the AI Era

- (Why Large Graph Processing design still require humans)
-  **What AI cannot decide for you (yet)**
 - Defining the β value in Page Rank Algorithm
 - If you are ranking scientific papers, you might want a higher β because citations are valuable information we want to give more credits.
 - If you are ranking real-time news, you might want a lower β because you want to reward what is currently being linked to, rather than the historical authority of a site.
 - Convergence Criteria (ϵ value) vs. Compute Budget
 - Is a precision of 0.000001 necessary, or is 0.001 "good enough" for your recommendation engine?
 - Only a human can decide when the marginal gain in accuracy is no longer worth the thousands of dollars in extra compute time.



- **What AI can assist you**

- Writing the compute function
- Memory Management for Large States
 - AI can help configure **checkpointing intervals** (how often to save to disk)

Acknowledgements

- CS4225 slides by He Bingsheng and Bryan Hooi
- Claudia Hauff - Big Data Processing
(https://chauff.github.io/documents/bdp/graph_giraph.pdf)
mmds.org – Mining of Massive Datasets - Jure Leskovec, Anand Rajaraman, Jeff Ullman
- https://jakobmikschi.eu/post/openstreetmap_routing/
- <https://www.edrawmax.com/online-map-maker.html>
- <https://learnforeverlearn.com/pagerank/>
- <https://neo4j.com/>